Elektrotehnički fakultet
Univerzitet u Banjoj Luci

IZVJEŠTAJ PROJEKTNOG ZADATKA

iz predmeta

SISTEMI ZA DIGITALNU OBRADU SIGNALA

Student:
Čiča Danilo 1129/18

Mentori:
prof. dr Mladen Knežić
prof. dr Mitar Simić
ma Vedran Jovanović
dipl. inž. Dimitrije Obradović

Januar 2026. godine

1. Opis projektnog zadatka

Cilj projektnog zadatka je implementacija sistema za kompresiju grayscale slika zasnovanog na diskretnoj kosinusnoj transformaciji (DCT II) koristeći razvojno okruženje ADSP-21489 te zatim dekompresiju istih na računaru pomoću Python jezika. Pored same implementacije, potrebno je izvršiti i optimizaciju koda s ciljem smanjenja vremena izvršavanja i zauzeća memorije. Osnovni tok obrade slike i ono što je potrebno realizovati:

- Učitavanje RGB slike na razvojnoj platformi i njena konverzija u grayscale
- Centriranje vrijednosti piksela oko nule i segmentiranje slike na blokove dimenzija N × N
- Primjena dvodimenzionalne DCT II na svaki blok
- Kvantovanje DCT koeficijenata pomoću odabrane matrice kvantizacije
- Cik-cak (zig-zag) permutacija koeficijenata radi grupisanja niskih i visokih frekvencija
- Kodovanje DC koeficijenta kao razlika u odnosu na prethodni blok i kodovanje AC koeficijenata *RLE(run-length)* metodom
- Serijalizacija rezultata u proizvoljnom formatu
- Dekompresija i analiza rezultata u Python-u ili MATLAB-u, prikazati metrike kao što su *MSE(Mean Squared Error)* i *PSNR(Peak Signal-to-Noise Ratio)* u odnosu na originalnu sliku

2. Izrada projektnog zadatka

Podatke o slici ćemo na DSP platformu učitati pomoću *image.bmp.h* header fajla generisanog skriptom *generateHeader.py*. Header fajl sadrži dimenzije slike te unaprijed inicijalizovan niz RGB piksela i deklarisan niz za grayscale piksele smještene u odgovarajuće memorijske sekcije. Ovim dimenzijama i nizovima ćemo pristupiti u svom CrossCore Embedded projektu tako što ćemo samo uključiti .h fajl koji treba da se nalazi u *Project/Debug* u naš glavni .c fajl.

U ovom projektu ćemo rgb i grayscale nizove smjestiti na sledeći način :

```
unsigned char gray[90000] __attribute__((section("seg_sdram1")));

const unsigned char rgb[270000] __attribute__((section("seg_sdram2")))
```

Slika 2.1 – *Mapiranje nizova rgb i gray*

Dok će *app.ldf* skripta koja mapira memorijske segmente u našem projektu, izgledati ovako:

```
mem_block3_dm32        { TYPE(DM RAM) START(0x000E0000) END(0x000E7FFF) WIDTH(32) }
mem_sram               { TYPE(DM RAM ASYNCHRONOUS) START(0x0C000000) END(0x0C07FFFF) WIDTH(16) }
mem_flash              { TYPE(DM RAM ASYNCHRONOUS) START(0x04000000) END(0x043FFFFF) WIDTH(8) }
mem_sdram1             { TYPE(DM RAM) START(0x00200000) END(0x003FFFFF) WIDTH(32) }  // bank0
mem_sdram2             { TYPE(DM RAM) START(0x00400000) END(0x005FFFFF) WIDTH(32) }  // bank1
mem_sdram3             { TYPE(DM RAM) START(0x00600000) END(0x007FFFFF) WIDTH(32) }  // bank2
mem_sdram4             { TYPE(DM RAM) START(0x00800000) END(0x009FFFFF) WIDTH(32) }  // bank3
```

Slika 2.2 – *Mapiranje memorijskih regiona na DSP*

```
  dxe_seg_sdram1
{
    INPUT_SECTIONS( $OBJECTS(seg_sdram1) )
} > mem_sdram1

  dxe_seg_sdram2
{
    INPUT_SECTIONS( $OBJECTS(seg_sdram2) )
} > mem_sdram2

 dxe_seg_sdram3
{
    INPUT_SECTIONS( $OBJECTS(seg_sdram3) )
} > mem_sdram3

  dxe_seg_pm_fast
{
    INPUT_SECTIONS( $OBJECTS(seg_pm_fast) )
} > mem_block2_pm32
```

Slika 2.3 – *Mapiranje korištenih memorijskih sekcija*

Dakle *linker* će smjestiti rgb i gray nizove u dva odvojena bloka SDRAM memorijskog regiona ovo radimo jer svaki od četiri bloka SDRAM ima odvojenu liniju za selekciju *(MS0–MS3)* te ovo doprinosi brzini upisivanja iz jednog u drugi niz.

Funkcija *rgb_to_grayscale* vrši upisivanje vrijednosti grayscale piksela u niz gray na osnovu kombinacije vrijednosti rgb piksela na sledeći način :

$$Gray = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \tag{2.1}$$

međutim da bismo prešli sa *float* aritmetike na cjelobrojnu aritmetiku uvešćemo optimizaciju :

$$Gray = (76 \cdot R + 150 \cdot G + 29 \cdot B + 128) >> 8 \tag{2.2}$$

Ovo radimo jer je ovo brža i pogodnija realizacija za DSP platformu.

Testiranjem dobijamo optimalne performanse kada ovo izračunavanje i upisivanje izvršavamo 6 piksela po iteraciji, ovo je realizovano ručnim odmotavanjem(*unroll*) petlje. Dodatno koristeći *restrict* deklaraciju za oba niza(rgb, gray) signaliziramo kompajleru da nema preklapanja(*alliasing*) između ova dva niza te mu to omogućava da bolje optimizuje petlju. Kao što je navedeno ova dva niza se nalaze u dva odvojena bloka SDRAM te za ovakav raspored dobijamo bolje performanse nego u slučaju njihovog smještanja u SRAM koji je najbrža eksterna memorija. Mogući razlog za ovo je podrška za SIMD režim rada preko 64-bitne *EPD*(*external port data bus*) koja postoji za SDRAM memoriju, te ovo omogućava dodatne optimizacije.

Funkcija *segment_and_stream_block_pipeline* vrši segmentaciju niza gray na blokove veličine 8x8 te centriranje vrijednosti unutar tih blokova oko nule, pri čemu u originalnoj slici idemo od lijevo prema desno i od gore prema dole pri segmentaciji na blokove. Prilikom segmentacije potrebno je uraditi dopunjavanje ivičnih blokova ukoliko visina ili širina slike nisu dijelilac broja 8 te da ne bismo provjeravali za svaki piksel da li je van opsega dijelimo upis tako da se ova provjera izvršava samo za ivične blokove i na taj način imamo manji ukupan broj poređenja. Ovi blokovi biće smješteni u brzu internu memoriju *mem_block1_dm32* i tu će se nalaziti kroz čitavu obradu sve do upisa rezultata obrade nazad u SDRAM memoriju. Nad svakim blokom odvojeno pozivamo prvo *dct_and_quant_block* te zatim rezultat toga prosljeđujemo u *encode_quant_block_to_text* gdje će se rezultat upisati u *textBlock*.

Funkcija *dct_and_quant_block* nad ulaznim blokom izvršava optimizovanu dvodimenzionalnu DCT transformaciju tako što množimo prvo po redovima pa onda po kolonama koristeći unaprijed izračunatu *DCT_C* matricu. Kvantizacija ovih koeficijenata se vrši njihovim množenjem sa unaprijed izračunatom inverznom kvantizacionom matricom *INV_Q*, čime se izbjegava upotreba operacije dijeljenja u floating-point aritmetici što je implementacija *OPTIMIZED_DIVISON* optimizacije. Obe unaprijed izračunate matrice se nalaze u *mem_block2_pm32* internoj brzoj memoriji. Odabrana je standardna matrica kvantizacije prema JPEG standardu za kompresiju.

```
// init DCT matrix C
for (int u = 0; u < DCT_N; ++u) {
    float alpha = (u == 0) ? sqrtf(1.0f / DCT_N) : sqrtf(2.0f / DCT_N);
    for (int x = 0; x < DCT_N; ++x) {
        DCT_C[u][x] = alpha * cosf((M_PI_F / (float)DCT_N) * ((float)x + 0.5f) * (float)u);
    }
}
```

Slika 2.4 – *Računanje DCT_C matrice*

```
// F = C * f * C^T

// row-wise transform: G[y][u] = sum of f[y][x] * C[u][x]
// G = f * C^T
for (int y = 0; y < DCT_N; ++y) {
    for (int u = 0; u < DCT_N; ++u) {
        float s = 0.0f;
        for (int x = 0; x < DCT_N; ++x) s += DCT_C[u][x] * f[y][x];
        G[y][u] = s;
    }
}

// column-wise transform: F[u][v] = sum of C[u][y] * G[y][v]
// F = C * G
for (int u = 0; u < DCT_N; ++u) {
    for (int v = 0; v < DCT_N; ++v) {
        float s = 0.0f;
        for (int y = 0; y < DCT_N; ++y) s += DCT_C[u][y] * G[y][v];
        F[u][v] = s;
    }
}
```

Slika 2.5 – *Optimizovana 2D DCT implementacija*

Funkcija *encode_quant_block_to_text* nad kvantizovanim 8×8 DCT blokom izvodi zig-zag permutaciju a zatim diferencijalno kodira DC koeficijent i primenjuje *RLE* kodiranje za AC koeficijente. Zatim upisujemo DC razliku, broj parova za AC, te same parove u obliku (*run*,*value*) u izlazni *textBlock*, na ovaj način implementiramo serijalizaciju rezultata u tekstualnom formatu što nam omogućava da vidimo tačno šta je sačuvano za svaki blok. Funkcija vraća pokazivač na sledeću slobodnu poziciju u *textBlock*-u. Tipično kompresija podrazumijeva čuvanje rezultata u binarnom obliku i to je ovde moguće uraditi i bez upisivanja rezultata obrade nazad u SDRAM, tako što svaki blok pišemo pomoću *fwrite*

direktno u serijalizacioni fajl. Ovde treba obratiti pažnju na to da ADSP-21489 može da adresira minimum 4 bajta te da su upisane veličine *char, short, int* tipova iste, ovo je važno prilikom čitanja fajla kod dekompresije.

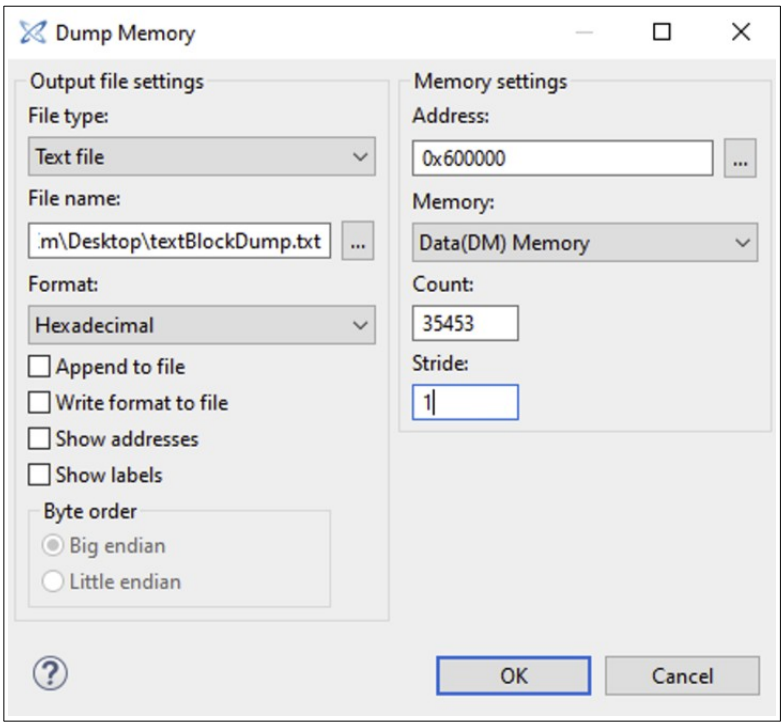
Dodatno još koristimo i pomoćne funkcije *init_dct_and_invq* koja unaprijed samo jednom računa *DCT_C* i *INV_Q* te *write_int* koja nam upisuje int vrijednosti u tekstualnom formatu.

Da bi dobili sam kompresovani fajl sadržaj *textBlock* :

```
#pragma section("seg_sdram3")
char textBlock[300000];
```

Slika 2.6 – *Izlazni sadržaj - textBlock*

je potrebno prebaciti nazad na računar, ovo radimo pomoću CrossCore Embedded(CCES) opcije *Memory Browser/Dump Memory* :



Slika 2.7 – *Dump memory prozor u okviru CCES*

*G*dje biramo *Hexadecimal* format a u polje *Count* unosimo broj karaktera koji je upisan u *textBlock* i čuvamo rezultat kao tekstualni fajl.

Performanse mjerimo pomoću mehanizama zaglavlja *cycle_count.h* koje nam omogućuje korišćenje metoda *START_CYCLE_COUNT*, *STOP_CYCLE_COUNT* i *PRINT_CYCLES*. Ove metode služe za dobijanje i ispisivanje informacija o broju ciklusa koje DSP utroši za izvršavanje dijelova koda između poziva *START_CYCLE_COUNT* i *STOP_CYCLE_COUNT.*

U main pozivamo samo dvije funkcije *rgb_to_grayscale* koja nam daje grayscale vrijednosti piksela i *segment_and_stream_block_pipeline* koja vrši obradu blokova od segmentiranja do upisa rezultata te ćemo posmatrati samo dva odvojena vremena izvršavanja. Broj procesorskih ciklusa koji dobijamo za testnu sliku *CityLandscape.bmp* koja je dimenzija 300x300 :

Bez kompajlerskih optimizacija :

- Grayscale cycles: 8 484 001
- Segmentation/Center/DCT/Quantization/Encoding cycles: 73 049 272

Sa optimizacijom za maksimalnu brzinu (*–Ov100* / *#pragma optimize_for_speed*):

- Grayscale cycles: 2 353 052
- Segmentation/Center/DCT/Quantization/Encoding cycles: 8 960 114

Sa još dodatno uključenim opcijama

- *Allow automatic SIMD code generation in linear code (-linear-simd)*

- *Allow automatic SIMD code generation in loops, aggregate assignments and inlined memcpy operations (-loop-simd)*

Imamo :

- Grayscale cycles: 2 352 975
- Segmentation/Center/DCT/Quantization/Encoding cycles: 7 035 109

Prema tome vidimo da je kompajler uspio da postigne ubrzanje od $\approx 3.6x$ za grayscale funkciju te od oko $\approx 10.3x$ za ukupnu obradu svih blokova.

Primjer izvršavanja :

```
Output
Loading application: "C:\Users\Em\Desktop\SDOS\SDOS_CCES\CompressGrayPicture\Debug\CompressGrayPicture.dxe"
Load complete.
Picture dimensions: 300 x 300
Grayscale cycles: 2353045
Segmentation/Center/DCT/Quantization/Encoding cycles: 7035144
Segmented into NBx=38, NBy=38, total blocks=1444, padded 304x304

textBlock - Encoding OK, Count=35453
```

Slika 2.8 – *Primjer izvršavanja u CCES okruženju*

Za *textBlockDump.txt* pozivamo python skriptu koja Heksadecimalni memory dump pretvara u naš kompresovani .txt fajl :

python memory_dump_to_compressed.py textBlockDump.txt out_compressed.txt

a zatim nad *out_compressed.txt* radimo dekompresiju :

python decompress_to_bmp.py out_compressed.txt outGray.bmp CityLandscapeGray.raw

Gdje je *outGray.bmp* izlazna grayscale slika, dok *CityLandscapeGray.raw* parametar predstavlja originalne piksele u odnosu na koje računamo *MSE/PSNR* dekompresovanih piksela. Originalne piksele nam generiše skripta *generateHeader.py* (zajedno sa *CityLandscape.bmp.h* i *CityLandscapeGray.bmp*) .

Primjer dekompresije:

```
C:\Users\Em\Desktop>python decompress_to_bmp.py out_compressed.txt outGray.bmp CityLandscapeGray.raw
Parsing: out_compressed.txt
Header: width=300 height=300 NBx=38 NBy=38 num_blocks=1444
Saved grayscale image: outGray.bmp
MSE over 90000 pixels: 15.3587
PSNR: 36.267 dB
Done.

C:\Users\Em\Desktop>
```

Slika 2.9 – *Primjer dekompresije*

Vizuelnim poređenjem *CityLandscapeGray.bmp i outGray.bmp* možemo da zaključimo da li je kvalitet dekompresovane slike zadovoljavajući za različite svrhe korištenja.

CompressRGBPicture je primjer realizacije kompresije potpune rgb slike jednostavnom modifikacijom prethodne obrade na način tako da umjesto *rgb_to_grayscale* funkcije koristimo *rgb_split* te na taj način dijelimo rgb niz na r, g i b komponente. A zatim vršimo istu *segment_and_stream_block_pipeline* obradu koju smo radili na gray na sva tri niza. Zbog jednostavnosti rezultate upisujemo u tri odvojena tekstualna bloka *textBlockR, textBlockG, textBlockB* koji će se serijalizovati u 3 odvojena fajla kompresije. Python skripte *generateHeaderRGB.py*, *decompress_to_rgb_bmp.py* su adekvatne modifikacije originalnih skripti korištenih u projektu za kompresiju/dekompresiju grayscale slike.

Primjer izvršavanja :

```
Loading application: "C:\Users\Em\Desktop\SDOS\SDOS_CCES\CompressRGBPicture\Debug\CompressRGBPicture.dxe"
Load complete.
Picture dimensions: 300 x 300
Split cycles: 4475977
R total cycles: 7101164
textBlockR USED = 37775
Segmented into NBx=38, NBy=38, total blocks=1444, padded 304x304

G total cycles: 7020581
textBlockG USED = 34958
Segmented into NBx=38, NBy=38, total blocks=1444, padded 304x304

B total cycles: 6971011
textBlockB USED = 33280
Segmented into NBx=38, NBy=38, total blocks=1444, padded 304x304
```

Slika 2.10 – *Primjer izvršavanja RGB kompresije u CCES*

Primjer dekompresije :

```
C:\Users\Em\Desktop>python decompress_to_rgb_bmp.py outCompressedR.txt outCompressedG.txt outCompressedB.txt outRGB.bmp CityLandscapeRGB.raw
Saved: outRGB.bmp
MSE R: 15.7604
MSE G: 15.3300
MSE B: 15.2256
MSE total: 15.4387
PSNR: 36.245 dB

C:\Users\Em\Desktop>
```

Slika 2.11 – *Primjer dekompresije RGB slike*

Te možemo vizuelno da uporedimo *outRGB.bmp* i *CityLandscapeRGB.bmp.*

3. Zaključak

Kod evaluacije performansi smo mogli da vidimo da kompajler značajno bolje može da optimizuje izvršavanje obrade nad nekim podacima ukoliko se ti podaci nalaze u brzoj internoj memoriji nasuprot eksternoj memoriji. Prema tome potrebno je pokušati eliminisati upisivanje i obradu podataka u eksternoj memoriji kad god je to moguće. Mi u našem projektu imamo tri sekcije u SDRAM memoriji *rgb* i *gray* nizove na ulazu te *textBlock* na izlazu, ovo bismo mogli potencijalno da svedemo na čuvanje samo *rgb* pri početnom učitavanju. Tako što bi nakon učitavanja rgb piksela radili prvo segmentaciju na blokove te zatim svaki blok učitavali u internu memoriju i propuštali kroz čitavu obradu grayscale/centriranje/DCT/Kvantizacija/Kodovanje i zatim direktno upisivali rezultate binarnim *fwrite* u kompresovani fajl. Kao što smo već spomenuli ovde je neophodno voditi računa o adresabilnosti (minimum 4 bajta) na našoj platformi, ovde bismo mogli optimizovati memorijsko zauzeće pakovanjem više podataka u jedan 4 bajtni tip (npr. 4 *char* vrijednosti zajedno).

4. Literatura

[1] *Analog Devices ADSP-21489 Data Sheet*